Python: exercises on files

This notebook was generated in **pseudocode** mode.

Exercise 1: Writing and Reading a File

Exercise Description

Write a program that:

1. Creates and opens a file named `"students.txt"` in write mode.
2. Writes the names of five students, each on a new line, to the file.
3. Closes the file.
4. Reopens the file in read mode and prints each student's name.

Store the result in a variable `result` containing a list of the student names read from the file.

Your solution

```
# step 1: open the file "students.txt" in write mode using with statement
    # step 2: write each student name on a new line
        # step 3: write "Alice" followed by newline
        # step 4: write "Bob" followed by newline
        # step 5: write "Charlie" followed by newline
        # step 6: write "David" followed by newline
        # step 7: write "Eve" followed by newline

# step 8: initialize result list to store student names
result = []
# step 9: open the file "students.txt" in read mode using with statement
    # step 10: iterate through each line in the file
        # step 11: strip whitespace from the line and append to result list
        # step 12: print the stripped line
```

Test your solution

```
# Test that result contains the expected student names
assert len(result) == 5
assert "Alice" in result
assert "Bob" in result
assert "Charlie" in result
assert "David" in result
assert "Eve" in result
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Storing and Loading Scores

Exercise Description

1. Create a list of five scores: `[85, 90, 78, 92, 88]`.
2. Write a program that:
    - Opens a file named `"scores.txt"` in write mode.
    - Writes each score from the list to the file, with each score on a new line.
    - Closes the file.
3. Reopen the file in read mode and:
    - Read all lines from the file, convert each line to an integer, and store the results in a new list.
    - Store the loaded scores in a variable `loaded_scores`.

Your solution

```
# step 1: create a list of scores with the given values
scores = [85, 90, 78, 92, 88]

# step 2: open the file "scores.txt" in write mode using with statement
```

```
    # step 3: iterate through each score in the scores list
        # step 4: write the score followed by a newline to the file

# step 5: initialize an empty list to store loaded scores
loaded_scores = []
# step 6: open the file "scores.txt" in read mode using with statement
    # step 7: iterate through each line in the file
        # step 8: strip whitespace from the line and convert to integer
        # step 9: append the integer to the loaded_scores list

# step 10: print the list of loaded scores
```

Test your solution

```
# Test that loaded_scores matches the original scores
assert loaded_scores == [85, 90, 78, 92, 88]
assert len(loaded_scores) == 5
assert all(isinstance(score, int) for score in loaded_scores)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Appending to a File

Exercise Description

Write a program that:

1. Opens a file named `"notes.txt"` in append mode.
2. Adds three predefined notes to the file: "First note", "Second note", "Third note".
3. Each note should be written on a new line.
4. Finally, opens the file in read mode and stores all notes in a variable `all_notes` as a list.

Store the result in a variable `all_notes` containing a list of all the notes read from the file.

Your solution

```
# step 1: define the three notes to be added
notes = ["First note", "Second note", "Third note"]

# step 2: open the file "notes.txt" in append mode using with statement
    # step 3: iterate through each note in the notes list
        # step 4: write the note followed by a newline to the file

# step 5: initialize an empty list to store all notes
all_notes = []
# step 6: open the file "notes.txt" in read mode using with statement
    # step 7: print "All notes:" as a header
```

```
    # step 8: iterate through each line in the file
        # step 9: strip whitespace from the line
        # step 10: append the stripped line to all_notes list
        # step 11: print the stripped line
```

Test your solution

```
# Test that all_notes contains the expected notes
assert "First note" in all_notes
assert "Second note" in all_notes
assert "Third note" in all_notes
assert len(all_notes) >= 3  # May contain more if file existed before
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Counting Words in a File

Exercise Description

Write a program that:

1. Uses the predefined sentence: "Python is a powerful programming language".
2. Opens a file named `"sentence.txt"` in write mode and writes the sentence to the file.
3. Closes the file.
4. Reopens the file in read mode, reads the sentence back, and counts the number of words.
5. Stores the word count in a variable `word_count`.

Hint: Use `split()` on the sentence to count the words.

Your solution

```
# step 1: define the sentence to be written to the file
sentence = "Python is a powerful programming language"

# step 2: open the file "sentence.txt" in write mode using with statement
    # step 3: write the sentence to the file

# step 4: open the file "sentence.txt" in read mode using with statement
    # step 5: read the whole sentence from the file

# step 6: count the words by splitting the sentence into a list of words
# step 7: get the length of the split sentence to count words
# step 8: print the word count and the split words for verification
```

Test your solution

```
# Test that word_count is correct
assert word_count == 6  # "Python is a powerful programming language" has 6 words
```

```
assert isinstance(word_count, int)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Saving and Loading a Shopping List

Exercise Description

Write a program that:

1. Uses a predefined shopping list: `["apple", "bread", "milk"]`.
2. Opens a file named `"shopping_list.txt"` in write mode and writes each item from the list to a new line.
3. Reopens the file in read mode and:
    - Reads all lines into a list.
    - Stores the shopping list from the file in a variable `loaded_shopping_list`.

Your solution

```
# step 1: define the shopping list with predefined items
shopping_list = ["apple", "bread", "milk"]

# step 2: open the file "shopping_list.txt" in write mode using with statement
    # step 3: iterate through each item in the shopping list
        # step 4: write the item followed by a newline to the file

# step 5: initialize an empty list to store loaded shopping list
loaded_shopping_list = []
# step 6: print "Shopping List from File:" as a header
# step 7: open the file "shopping_list.txt" in read mode using with statement
    # step 8: iterate through each line in the file
        # step 9: strip whitespace from the line
        # step 10: append the stripped line to loaded_shopping_list
        # step 11: print the stripped line
```

Test your solution

```
# Test that loaded_shopping_list matches the original
assert loaded_shopping_list == ["apple", "bread", "milk"]
assert len(loaded_shopping_list) == 3
assert "apple" in loaded_shopping_list
assert "bread" in loaded_shopping_list
assert "milk" in loaded_shopping_list
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Saving Calculated Results to a File

Exercise Description

1. Create a list of numbers: `[5, 10, 15, 20]`.
2. Write a program that:
    - Calculates the square of each number and writes the results to a file named `"squares.txt"`.
    - Each square should be written on a new line.
3. Open the file in read mode and store each square in a variable `squares_list` as a list of integers.

Your solution

```
# step 1: define the list of numbers
numbers = [5, 10, 15, 20]

# step 2: open the file "squares.txt" in write mode using with statement
    # step 3: iterate through each number in the numbers list
```

```
        # step 4: calculate the square of the number using exponentiation
        # step 5: write the square followed by a newline to the file

# step 6: initialize an empty list to store squares
squares_list = []
# step 7: open the file "squares.txt" in read mode using with statement
    # step 8: print "Squares:" as a header
    # step 9: iterate through each line in the file
        # step 10: strip whitespace from the line and convert to integer
        # step 11: append the integer to squares_list
        # step 12: print the stripped line
```

Test your solution

```
# Test that squares_list contains the correct squares
assert squares_list == [25, 100, 225, 400]  # 5^2, 10^2, 15^2, 20^2
assert len(squares_list) == 4
assert all(isinstance(square, int) for square in squares_list)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Reversing Lines in a File

Exercise Description

Write a program that:

1. Uses five predefined lines of text: `["Line 1", "Line 2", "Line 3", "Line 4", "Line 5"]`.
2. Saves each line to a file named `"reversed_lines.txt"`.
3. Opens the file in read mode and prints each line in reverse order.
4. Stores the reversed lines in a variable `reversed_lines` as a list.

Your solution

```
# step 1: define the list of lines to be written
lines = ["Line 1", "Line 2", "Line 3", "Line 4", "Line 5"]

# step 2: open the file "reversed_lines.txt" in write mode using with statement
    # step 3: iterate through each line in the lines list
        # step 4: write the line followed by a newline to the file

# step 5: open the file "reversed_lines.txt" in read mode using with statement
    # step 6: read all lines from the file into a list
    # step 7: print "Reversed Lines:" as a header
    # step 8: iterate through the lines in reverse order using reversed() function
        # step 9: strip whitespace from each line
```

```
        # step 10: append the stripped line to reversed_lines list
        # step 11: print the stripped line
```

Test your solution

```
# Test that reversed_lines contains lines in reverse order
assert reversed_lines == ["Line 5", "Line 4", "Line 3", "Line 2", "Line 1"]
assert len(reversed_lines) == 5
assert reversed_lines[0] == "Line 5"
assert reversed_lines[-1] == "Line 1"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Finding the Longest Word in a File

Exercise Description

Write a program that:

1. Uses the predefined sentence: "Programming with Python is incredibly rewarding".
2. Writes the sentence to a file named `"sentence.txt"`.
3. Opens the file in read mode, reads the sentence back, and finds the longest word.
4. Stores the longest word in a variable `longest_word`.

Your solution

```
# step 1: define the sentence to be written to the file
sentence = "Programming with Python is incredibly rewarding"

# step 2: open the file "sentence.txt" in write mode using with statement
    # step 3: write the sentence to the file

# step 4: open the file "sentence.txt" in read mode using with statement
    # step 5: read the sentence from the file and split into words
    # step 6: find the longest word using max() function with key=len

# step 7: print the longest word
```

Test your solution

```
# Test that longest_word is correct
assert longest_word == "Programming"  # "Programming" is the longest word (11 charact
assert len(longest_word) == 11
assert isinstance(longest_word, str)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Reading Data and Calculating the Average

Exercise Description

1. Create a file named `"data.txt"` containing numerical values: `[10, 15, 20, 25, 30]`, each on a new line.
2. Write a program that:
    - Reads each value from the file.
    - Calculates the average of the values.
3. Store the calculated average in a variable `average`.

Your solution

```
# step 1: define the data values to be written to the file
data_values = [10, 15, 20, 25, 30]

# step 2: open the file "data.txt" in write mode using with statement
    # step 3: iterate through each value in data_values
        # step 4: write the value followed by a newline to the file
```

```
# step 5: initialize an empty list to store values read from file
values = []
# step 6: open the file "data.txt" in read mode using with statement
    # step 7: iterate through each line in the file
        # step 8: strip whitespace from the line and convert to integer
        # step 9: append the integer to the values list

# step 10: calculate the average by dividing sum of values by length of values
# step 11: print the calculated average
```

Test your solution

```
# Test that average is calculated correctly
assert average == 20.0  # (10+15+20+25+30)/5 = 100/5 = 20.0
assert isinstance(average, float)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Saving a List of Names in Uppercase

Exercise Description

Write a program that:

1. Uses five predefined names: `["alice", "bob", "charlie", "diana", "eve"]`.
2. Saves each name in uppercase to a file named `"uppercase_names.txt"`, with each name on a new line.
3. Opens the file in read mode and stores each uppercase name in a variable `uppercase_names` as a list.

Your solution

```
# step 1: define the list of names in lowercase
names = ["alice", "bob", "charlie", "diana", "eve"]

# step 2: open the file "uppercase_names.txt" in write mode using with statement
    # step 3: iterate through each name in the names list
        # step 4: convert the name to uppercase using upper() method
        # step 5: write the uppercase name followed by a newline to the file

# step 6: initialize an empty list to store uppercase names
uppercase_names = []
# step 7: open the file "uppercase_names.txt" in read mode using with statement
    # step 8: print "Names in uppercase:" as a header
    # step 9: iterate through each line in the file
        # step 10: strip whitespace from the line
```

```
        # step 11: append the stripped line to uppercase_names list
        # step 12: print the stripped line
```

Test your solution

```
# Test that uppercase_names contains the correct uppercase names
assert uppercase_names == ["ALICE", "BOB", "CHARLIE", "DIANA", "EVE"]
assert len(uppercase_names) == 5
assert all(name.isupper() for name in uppercase_names)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Writing and Checking for Palindromes

Exercise Description

Write a program that:

1. Uses five predefined words: `["racecar", "hello", "level", "python", "radar"]`.
2. Saves each word to a file named `"words.txt"`.
3. Reopens the file, reads each word, and stores only those words that are palindromes (words that read the same forwards and backwards) in a variable `palindromes` as a list.

Your solution

```
# step 1: define the list of words to be written to the file
words = ["racecar", "hello", "level", "python", "radar"]

# step 2: open the file "words.txt" in write mode using with statement
    # step 3: iterate through each word in the words list
        # step 4: write the word followed by a newline to the file

# step 5: initialize an empty list to store palindromes
palindromes = []
# step 6: print "Palindromes:" as a header
# step 7: open the file "words.txt" in read mode using with statement
    # step 8: iterate through each line in the file
        # step 9: strip whitespace from the line to get the word
        # step 10: check if the word equals its reverse using slicing [::-1]
            # step 11: if it's a palindrome, append to palindromes list
            # step 12: print the palindrome word
```

Test your solution

```python
# Test that palindromes contains only the palindromic words
assert set(palindromes) == {"racecar", "level", "radar"}  # Only palindromes
assert len(palindromes) == 3
assert "hello" not in palindromes  # Not a palindrome
assert "python" not in palindromes  # Not a palindrome
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Counting Specific Words in a Text File

Exercise Description

1. Create a file named `"story.txt"` containing a short paragraph with predefined text.
2. Write a program that:
    - Uses the word "the" as the target word to count.
    - Reads the contents of the file and counts how many times the word appears in the text (case-insensitive).
3. Store the total count in a variable `word_count`.

Your solution

```
# step 1: define the story text as a list of sentences
story_text = ["Once upon a time in a land far, far away there was a brave knight.",
              "The knight set out on an adventure to rescue the kingdom."]

# step 2: open the file "story.txt" in write mode using with statement
    # step 3: iterate through each sentence in story_text
        # step 4: write the sentence followed by a newline to the file

# step 5: define the word to count (convert to lowercase for case-insensitive search)
word_to_count = "the"

# step 6: initialize count to 0
count = 0
# step 7: open the file "story.txt" in read mode using with statement
    # step 8: iterate through each line in the file
        # step 9: convert the line to lowercase and split into words
        # step 10: count occurrences of word_to_count in the words list
        # step 11: add the count to the total count

# step 12: print the final count with a formatted message
```

Test your solution

```
# Test that word_count is correct
assert word_count == 2  # "the" appears twice in the story ("The knight" and "the kin
assert isinstance(word_count, int)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: Writing a Simple CSV File

Exercise Description

Write a program that:

1. Uses predefined information for three people: `[("Alice", "25", "alice@email.com"), ("Bob", "30", "bob@email.com"), ("Charlie", "35", "charlie@email.com")]`.
2. Writes this information to a file named `"people.csv"`, with each field separated by commas and each person on a new line.
3. Includes a header row: "Name,Age,Email".
4. Opens the file in read mode and stores each line in a variable `csv_lines` as a list.

Your solution

```
# step 1: define the people data as a list of tuples
people_data = [("Alice", "25", "alice@email.com"), ("Bob", "30", "bob@email.com"), ('

# step 2: open the file "people.csv" in write mode using with statement
    # step 3: write the header row with column names
    # step 4: iterate through each person tuple in people_data
        # step 5: extract name, age, and email from the tuple
        # step 6: write the person's data in CSV format (comma-separated) with newli

# step 7: initialize an empty list to store CSV lines
csv_lines = []
# step 8: open the file "people.csv" in read mode using with statement
    # step 9: print "CSV Content:" as a header
    # step 10: iterate through each line in the file
        # step 11: strip whitespace from the line
        # step 12: append the stripped line to csv_lines list
        # step 13: print the stripped line
```

Test your solution

```
# Test that csv_lines contains the correct CSV content
assert len(csv_lines) == 4  # Header + 3 people
assert csv_lines[0] == "Name,Age,Email"  # Header
assert "Alice,25,alice@email.com" in csv_lines
assert "Bob,30,bob@email.com" in csv_lines
assert "Charlie,35,charlie@email.com" in csv_lines
```

Debug your solution